

Serialize, Unserialize, JSON Encode & Decode, Array Utility Functions

- Md. Morshedul Arefin

Serialize and Unserialize

Associative Array to String - 2 Ways:

1. Serialize
2. JSON - better

Serialize

```
$studentInfo = [  
    'firstName' => 'Smith',  
    'lastName' => 'Cooper',  
    'class' => 9,  
    'age' => 35  
];  
echo serialize($studentInfo);
```

Output:

```
a:4:{s:9:"firstName";s:9:"Smith";s:8:"lastName";s:6:"Cooper";s:5:"class";i:9;s:3:"age";i:35;}
```

Unserialize

```
$data =  
'a:4:{s:9:"firstName";s:9:"Smith";s:8:"lastName";s:6:"Cooper";s:5:"class";i:9;s:3:"age";i:35;}';  
print_r(unserialize($data));
```

JSON Encode

```
$studentInfo = [  
    'firstName' => 'Smith',  
    'lastName' => 'Cooper',  
    'class' => 9,  
    'age' => 35  
];  
echo json_encode($studentInfo);
```

Output:

```
{"firstName":"Smith","lastName":"Cooper","class":9,"age":35}
```

JSON Decode - Return stdClass object

```
$data = '{"firstName":"Smith","lastName":"Cooper","class":9,"age":35}';  
$result = json_decode($data);  
echo $result->firstName; // Smith  
print_r($result);
```

Output:

stdClass Object

```
(  
    [firstName] => Smith  
    [lastName] => Cooper  
    [class] => 9  
    [age] => 35  
)
```

JSON Decode - Return Array Again

```
$data = '{"firstName":"Smith","lastName":"Cooper","class":9,"age":35}';  
$result = json_decode($data,true);  
echo $result['firstName']; // Smith  
print_r($result);
```

Output:

Array

```
(  
    [firstName] => Smith  
    [lastName] => Cooper  
    [class] => 9  
    [age] => 35  
)
```

Array Utility Function - `array_walk(array,callback)`

- Executing a specific callback function on every item in an array sequentially.
- Modifying the original array's values directly by passing the value parameter by reference (&\$value).
- Providing access to both the element's value and its key/index within the callback function.
- Passing an optional external parameter into the callback function for additional context.

Example - Processing Array Elements

```
$colors = ['red', 'green', 'blue'];  
array_walk($colors, function ($value) {  
    echo "Color: ".$value."<br>";  
});
```

/*

Output:

Color: red

Color: green

Color: blue

*/

Example - In Place Modification

```
$prices = [100, 200, 300];
```

```
// Using &$price modifies the original array values directly  
array_walk($prices, function (&$price) {  
    $price *= 1.10; // Apply 10% tax  
});  
print_r($prices);
```

```
/* Output:
```

```
Array
```

```
(
```

```
    [0] => 110
```

```
    [1] => 220
```

```
    [2] => 330
```

```
)
```

```
*/
```

Example - Accessing Keys and Values

```
$capitals = [ 'Bangladesh' => 'Dhaka', 'Japan' => 'Tokyo' ];  
// First argument is value ($city), second is key ($country)  
array_walk($capitals, function ($city, $country) {  
    echo "The capital of {$country} is {$city}.\n";  
});
```

/* Output:

The capital of Bangladesh is Dhaka.

The capital of Japan is Tokyo.

*/

Example - Injecting Extra Data

```
$scores = [80, 90, 75];  
$bonus = 5;  
// Passing $bonus as the 3rd argument to array_walk() populates $extraPoints  
array_walk($scores, function (&$score, $key, $extraPoints) {  
    $score += $extraPoints;  
}, $bonus);  
print_r($scores);  
  
/*  
Output:  
Array ( [0] => 85 [1] => 95 [2] => 80 )  
*/
```

Array Utility Function - `array_map(callback,array)`

- `array_map()` applies a callback function to the elements of one or more arrays and returns a new array containing the transformed values.
- Unlike `array_walk()`, it never modifies the original array directly.

Example - Basic Transformation (Creates a New Array)

```
$numbers = [1, 2, 3, 4, 5];  
// Returns a new array with squared values  
$squares = array_map(function ($n) {  
    return $n * $n;  
}, $numbers);  
print_r($squares);
```

```
/*
```

Output:

```
Array ( [0] => 1, [1] => 4, [2] => 9, [3] => 16, [4] => 25 )
```

```
*/
```

```
// $numbers remains original: [1, 2, 3, 4, 5]
```

Example - Processing Multiple Arrays Parallely

```
$firstNames = ['John', 'Jane', 'Alice'];  
$lastNames  = ['Doe', 'Smith', 'Jones'];  
$fullNames = array_map(function ($first, $last) {  
    return "{$first} {$last}";  
}, $firstNames, $lastNames);  
print_r($fullNames);
```

```
/*
```

Output:

Array

(

[0] => John Doe

[1] => Jane Smith

[2] => Alice Jones

)

```
*/
```

Example - Combining/Zippping Arrays

```
$keys  = ['id', 'name', 'role'];  
$values = [101, 'Bob', 'Admin'];  
$zipped = array_map(null, $keys, $values);  
print_r($zipped);
```

```
/*
```

Output:

Array

```
(  
    [0] => Array ( [0] => id,  [1] => 101 )  
    [1] => Array ( [0] => name, [1] => Bob )  
    [2] => Array ( [0] => role, [1] => Admin )  
)  
*/
```

Array Utility Function - `array_filter(array,callback)`

- `array_filter()` filters elements of an array using a callback function.
- It returns a new array containing only the elements for which the callback returns true, leaving the original array unchanged and preserving its original keys.

Example - Basic Value Filtering

```
$numbers = [1, 2, 3, 4, 5, 6];  
// Keeps only even numbers  
$evenNumbers = array_filter($numbers, function ($n) {  
    return $n % 2 === 0;  
});  
print_r($evenNumbers);  
/*
```

Output (Notice keys 1, 3, 5 are preserved)

```
Array ( [1] => 2 [3] => 4 [5] => 6 )  
*/
```

```
// Use array_values() if you need to re-index the array from 0  
$reIndexed = array_values($evenNumbers);
```

Example - Removing Falsy Values (No Callback)

```
$mixedData = ['Alice', '', 0, false, null, 'Bob', []];  
// Omitting the callback strips out all falsy values  
$cleanData = array_filter($mixedData);  
print_r($cleanData);
```

```
/*
```

Output:

Array

```
(
```

```
    [0] => Alice
```

```
    [5] => Bob
```

```
)
```

```
*/
```

Example - Filtering by Key and Value

```
$userRoles = [  
    'admin_1' => 'admin',  
    'guest_1' => 'subscriber',  
    'admin_2' => 'editor',  
    'guest_2' => 'subscriber'  
];  
// Filter by matching both key pattern and value  
$admins = array_filter($userRoles, function ($role, $username) {  
    return str_starts_with($username, 'admin') && $role !== 'subscriber';  
}, ARRAY_FILTER_USE_BOTH);  
print_r($admins);
```

```
/*
```

Output:

```
Array ( [admin_1] => admin [admin_2] => editor )
```

```
*/
```

Array Utility Function - `array_reduce(array,callback)`

- `array_reduce()` iteratively processes an array to reduce it to a single value (such as a number, string, array, or object) by running a callback function across every element.

Example - Summing Values (Numeric Reduction)

```
$numbers = [10, 20, 30, 40];
```

```
// Calculate total sum starting from an initial value of 0
```

```
$total = array_reduce($numbers, function ($carry, $item) {
```

```
    return $carry + $item;
```

```
}, 0);
```

```
echo $total;
```

```
// Output: 100
```

Example - Sum of array elements that are even

```
$numbers = [1,2,3,4,5,6,7,8,9];  
$total = array_reduce($numbers, function($old,$new) {  
    if($new % 2 == 0) {  
        return $old+$new;  
    } else {  
        return $old;  
    }  
});  
echo $total; // 20
```

Example - Flattening a Multidimensional Array

```
$matrix = [  
    [1, 2],  
    [3, 4],  
    [5, 6]  
];  
$flattened = array_reduce($matrix, function ($carry, $item) {  
    return array_merge($carry, $item);  
}, []);  
print_r($flattened);
```

/* Output:

Array

(

[0] => 1 [1] => 2 [2] => 3 [3] => 4 [4] => 5 [5] => 6

)

*/

Example - Grouping Data by Category

```
$products = [  
    ['name' => 'Laptop', 'category' => 'Electronics'],  
    ['name' => 'Shirt', 'category' => 'Apparel'],  
    ['name' => 'Phone', 'category' => 'Electronics'],  
];  
$grouped = array_reduce($products, function ($carry, $item) {  
    $category = $item['category'];  
    $carry[$category][] = $item['name'];  
    return $carry;  
}, []);  
print_r($grouped);
```

```
/*  
Output:  
Array (  
    [Electronics] => Array ( [0] => Laptop, [1] => Phone )  
    [Apparel]     => Array ( [0] => Shirt )  
)  
*/
```

Example - Get maximum value from an array

```
$numbers = [33, 44, 88, 22, 18, 36];  
$result = array_reduce($numbers, function($old,$new) {  
    if($new>$old) {  
        $max = $new;  
    } else {  
        $max = $old;  
    }  
    return $max;  
});  
echo $result; // 88
```

Arrow Function

- Arrow functions (fn) are compact single-line closures introduced in PHP 7.4 designed to simplify inline callback functions.
- fn Keyword: Replaces function.
- Implicit Return: Whatever comes after => is automatically returned. Do not use the return keyword.
- Single Expression Only: The body can only contain one expression (no multi-line blocks or {}).
- Auto-Capturing Scope: Outer variables are automatically captured by value. no use (\$var) required.

Example - Basic Transformation

```
$numbers = [1, 2, 3, 4];
```

```
// Traditional closure
```

```
$squaredOld = array_map(function ($n) {  
    return $n * $n;  
}, $numbers);
```

```
// Arrow function equivalent
```

```
$squaredNew = array_map(fn($n) => $n * $n, $numbers);
```

Example - Automatic Outer Scope Access

```
$factor = 5;
```

```
$prices = [10, 20, 30];
```

```
// $factor is automatically captured from the parent scope
```

```
$multiplied = array_map(fn($p) => $p * $factor, $prices);
```

```
print_r($multiplied); // [50, 100, 150]
```

Real-World Example: E-Commerce Order Calculation

```
$cart = [  
    ['product' => 'Wireless Mouse', 'price' => 25.00, 'qty' => 2, 'in_stock' => true],  
    ['product' => 'Mechanical Keyboard', 'price' => 100.00, 'qty' => 1, 'in_stock' =>  
false], // Out of stock  
    ['product' => 'USB-C Cable', 'price' => 10.00, 'qty' => 3, 'in_stock' => true],  
    ['product' => '27" Monitor', 'price' => 200.00, 'qty' => 1, 'in_stock' => true],  
];
```

```
$taxRate = 0.10; // 10% tax
```

```
// Step 1: Filter out items that are not in stock
```

```
$inStockItems = array_filter($cart, fn($item) => $item['in_stock']);
```

Real-World Example: E-Commerce Order Calculation ..

```
// Step 2: Map each item to its total price including tax ( price * qty * 1.10 )  
$itemTotals = array_map(fn($item) => ($item['price'] * $item['qty']) * (1 + $taxRate),  
$inStockItems);
```

```
// Step 3: Reduce all individual totals into a single grand total balance  
$grandTotal = array_reduce($itemTotals, fn($carry, $cost) => $carry + $cost, 0.0);
```

```
echo "Grand Total: $" . number_format($grandTotal, 2); // Output: Grand Total:  
$341.00
```